AI CatalyESt

Complete Build Manual

From Zero to a Live Dashboard — Step by Step

Organisation	Siemens Technology India — Engineering Systems
Author	Vigneshvar SA
Live URL	https://ai-catalyest.onrender.com
Repository	github.com/vigneshvar11/ai-catalyst
Generated	May 10, 2026

This manual is designed so that anyone — even with zero coding experience — can understand how this dashboard was built and recreate it from scratch.

Table of Contents

1 Understanding the Project

What is AI CatalyESt?

AI CatalyESt (where **ES** stands for **Engineering Systems**) is a 12-month initiative at Siemens that teaches team members about Artificial Intelligence through hands-on monthly activities. Think of it as a structured learning journey — from basic AI concepts to building real AI solutions.

To track this journey, we built a **live dashboard** — a website that shows the schedule, scores, leaderboards, and lets the admin run quizzes and surveys in real-time.

■ Real-World Analogy

Think of the dashboard like a school's notice board — but digital and interactive. It shows who's winning, what's coming next, lets teachers (admins) create tests, and students can participate in live quizzes from their phones.

What Does the Dashboard Do?

- **Dashboard** — Shows countdown to next event, activity stats, progress overview
- **Leaderboard** — Ranks all 16 members by total points earned across 12 months
- **Calendar** — Visual 12-month plan with all events, dates, and phases
- **Team View** — Shows all members with their domains and roles
- **Live Quiz Engine** — Real-time quizzes with anti-cheat detection
- **Survey System** — Anonymous rating polls after presentations
- **Admin Panel** — Full control to manage members, events, points, quizzes

How Is It Built? — The Big Picture

Every website has three parts, just like a restaurant:

■■ The Restaurant Analogy

1. THE MENU (Frontend) — What the customer sees: the beautiful webpage with buttons, colors, and animations. Made with HTML, CSS, and JavaScript. 2. THE WAITER (Backend Server) — Takes orders from the customer and brings food from the kitchen. This is our Python Flask server — it receives requests and sends back data. 3. THE KITCHEN (Database) — Where all the ingredients (data) are stored. In production, our kitchen is MongoDB Atlas (a cloud database). For local development, we also have a simple JSON file called db.json as a fallback.

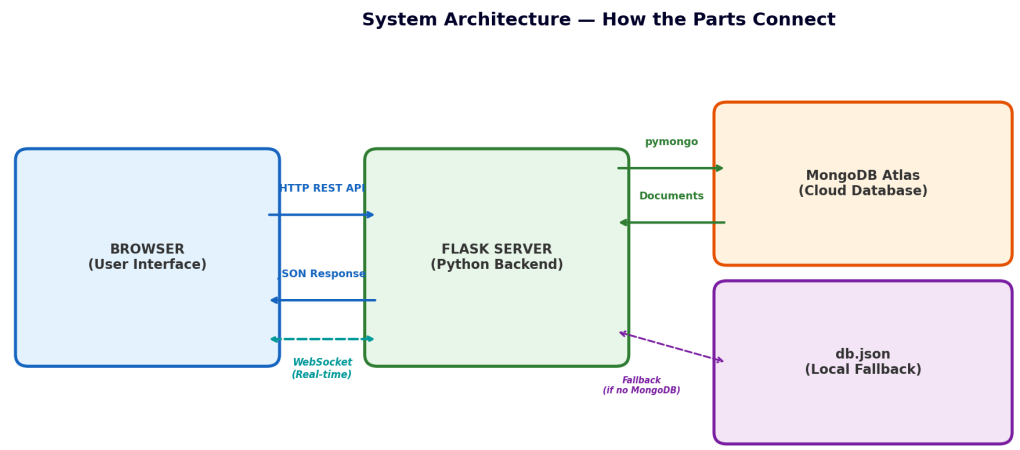

Figure 1.1 — System architecture showing how the browser, server, and database communicate.

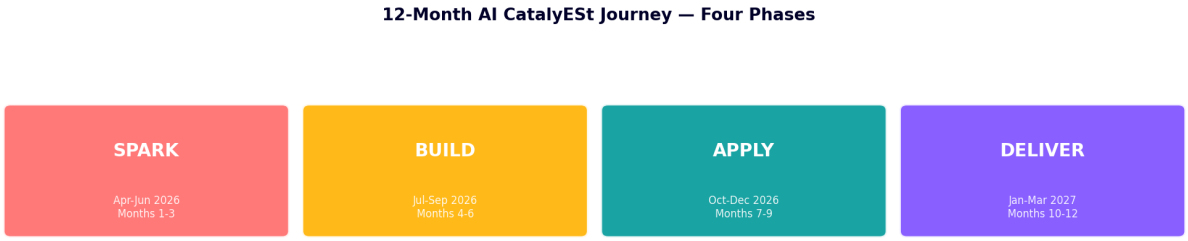

Figure 1.2 — The 12-month journey is divided into four progressive phases.

2 What You Need Before Starting

Before writing any code, you need to install a few free tools on your computer. Think of these as the **equipment** you need before you can start cooking.

1 Python 3.10+

The programming language our server is written in. HOW: Go to python.org/downloads and click the big yellow "Download" button. During installation, CHECK the box that says "Add Python to PATH". This is critical!

2 Visual Studio Code

The text editor where you'll write code (like Microsoft Word, but for code). HOW: Go to code.visualstudio.com and download it. It's free. Install normally.

3 Git

A version-control tool — think of it as an "undo history" for your entire project. HOW: Go to git-scm.com and download. Install with default settings — just keep clicking "Next".

4 Node.js (Optional)

Only needed if you want to run the JavaScript backend instead of Python. HOW: Go to nodejs.org and download the LTS version. Install normally.

5 A Modern Browser

Chrome, Edge, or Firefox to view your dashboard. HOW: You almost certainly already have one. Any modern browser works.

■ Checkpoint ✓

After installing everything, open a terminal (search "Terminal" or "Command Prompt" in your Start menu) and type these commands. If each shows a version number, you're ready!

TERMINAL

```
python --version     →  Python 3.12.3
git --version        →  git version 2.45.0
code --version       →  1.90.0
```

Creating Your Project Folder

Create a new folder on your computer. This is where ALL your project files will live. You can name it anything, but we'll call it **AI-CatalyESt**.

```
mkdir AI-CatalyESt
cd AI-CatalyESt
```

TERMINAL

Setting Up a Virtual Environment

■ What is a Virtual Environment?

Imagine you have a toolbox. A virtual environment is like having a SEPARATE toolbox for each project, so tools from one project don't interfere with another. It keeps your project's Python packages isolated and clean.

```
python -m venv .venv

# Activate it:
# On Windows:
.venv\Scripts\activate

# On Mac/Linux:
source .venv/bin/activate
```

TERMINAL

You'll know it's active when you see **(.venv)** at the beginning of your terminal prompt. Always activate this before working on the project!

3 Building the Database

Most websites use complex database systems like PostgreSQL or MongoDB. We started with something much simpler: a **single JSON file** for local development. Later, we upgraded to **MongoDB Atlas** (a free cloud database) for production, while keeping db.json as a fallback. This gives us the best of both worlds — easy debugging locally and reliable cloud storage in production.

> ■ **What is JSON?**
>
> JSON (JavaScript Object Notation) is a way to store data as text. It looks like a list of labeled boxes. For example: {"name": "Vigneshvar", "role": "Lead"}. It's human-readable — you can open it in any text editor and understand it!

Database Structure

Our database file **data/db.json** has 6 main collections (think of them as different drawers in a filing cabinet):

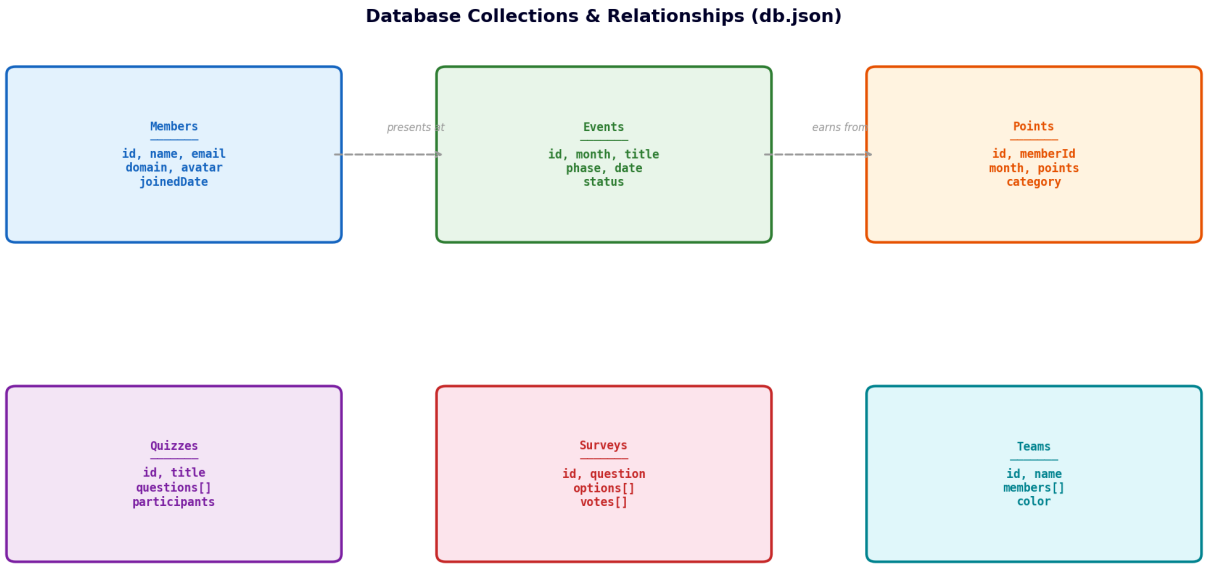

Figure 3.1 — The six data collections and how they relate to each other.

Step 1: Create the data folder and file

TERMINAL

```
mkdir data
# Create a new file: data/db.json
```

Step 2: Add the initial structure

Copy this into your db.json file. This is the skeleton of your database:

JSON

```
{
  "config": {
    "admin": { "username": "admin", "password": "admin" },
    "appName": "AI CatalyESt",
    "teamName": "Engineering Systems",
    "company": "Siemens"
  },
  "members": [],
  "events": [],
  "points": [],
  "quizzes": [],
  "surveys": [],
  "teams": []
}
```

Step 3: Add members

Each member is a JSON object with these fields. The **id** is generated from the name by making it lowercase and replacing spaces with hyphens:

JSON

```
{
  "id": "vigneshvar-sa",
  "name": "Vigneshvar SA",
  "email": "vigneshvar.sa@siemens.com",
  "role": "Initiative Lead",
  "domain": "PLM",
  "avatar": null,
  "joinedDate": "2026-04-01"
}
```

■ Key Concept: IDs

Every item in the database needs a unique identifier (ID). For members, we create IDs from their names (e.g., "Vigneshvar SA" → "vigneshvar-sa"). For other items like events or quizzes, we use a prefix + random characters (e.g., "event-a1b2c3d4").

Upgrading to MongoDB Atlas (Cloud Database)

While db.json works perfectly for local development, it has limitations in production: file writes can conflict under concurrent requests, and data resets on every Render deploy. To solve this, we added **MongoDB Atlas** — a free cloud-hosted database.

■■ What is MongoDB Atlas?

MongoDB is a NoSQL database that stores data as JSON-like "documents" — very similar to our db.json format! Atlas is MongoDB's free cloud service. Think of it as moving your filing cabinet from your office (local file) to a secure warehouse (cloud) that's always available and never loses data.

Setting Up MongoDB Atlas (Free M0 Tier)

1. **Create an Account**
   Go to mongodb.com/cloud/atlas and sign up for free. No credit card needed.

2. **Create a Cluster**
   Click "Build a Cluster" → Choose the FREE M0 tier → Pick a region close to your Render server.

3. **Create a Database User**
   Go to Security → Database Access → Add a user with read/write permissions. Save the password!

4. **Whitelist All IPs**
   Go to Security → Network Access → Add 0.0.0.0/0 to allow connections from Render.

5. **Get Connection String**
   Click "Connect" → "Connect your application" → Copy the connection string. Replace <password> with your database user's password.

How the Code Connects to MongoDB

The server uses a **lazy-initialization** pattern — it only connects to MongoDB when the first request arrives, not at startup. This avoids crashes if MongoDB is temporarily unavailable:

PYTHON

```python
import os
from pymongo import MongoClient
from pymongo.server_api import ServerApi

MONGODB_URI = os.environ.get("MONGODB_URI")  # From env var
_mongo_db = None  # Lazy-initialized

def _get_mongo():
    global _mongo_db
    if _mongo_db is None and MONGODB_URI:
        client = MongoClient(MONGODB_URI, server_api=ServerApi("1"))
        _mongo_db = client["aicatalyst"]
        # Auto-seed from db.json if database is empty
        if _mongo_db["members"].count_documents({}) == 0:
            seed = json.load(open(DB_PATH))
            for col in seed:
                if isinstance(seed[col], list) and seed[col]:
                    _mongo_db[col].insert_many(seed[col])
    return _mongo_db
```

■ Auto-Seeding Magic

When the server connects to MongoDB for the first time and finds it empty, it automatically copies all data from db.json into MongoDB. This means you never have to manually import data — just deploy and your cloud database is populated!

The updated **read_db()** and **write_db()** functions now try MongoDB first and fall back to the JSON file if MongoDB is not configured:

PYTHON

```python
def read_db():
    mongo = _get_mongo()
    if mongo:  # MongoDB available → use it
        db = {}
        for col in ["config","members","events","points",
                    "quizzes","surveys","teams"]:
            docs = list(mongo[col].find({}, {"_id": 0}))
            db[col] = docs[0] if col == "config" else docs
        return db
    # Fallback → read from db.json
    with open(DB_PATH) as f:
        return json.load(f)
```

4 Building the Backend Server

■■ The Waiter Analogy — Continued

Remember the restaurant? The backend server is the waiter. When you click a button on the website (place an order), the server receives it, goes to the database (kitchen), gets the data (food), and brings it back to your browser (table). Our waiter speaks Python and uses a framework called Flask.

Step 1: Install Flask

Flask is a Python library that turns your Python script into a web server. Install it along with the other packages we need:

TERMINAL

```
pip install flask flask-socketio gunicorn gevent gevent-websocket
pip install pymongo dnspython   # For MongoDB Atlas
```

Save these to a **requirements.txt** file so anyone can install them later:

TXT

```
flask>=3.0.0
flask-socketio>=5.3.0
gunicorn>=21.2.0
gevent>=24.2.1
gevent-websocket>=0.10.1
pymongo>=4.7.0
dnspython>=2.6.0
python-docx>=0.6.23
python-pptx>=0.6.23
Pillow>=10.0.0
```

Step 2: Create app.py — The Heart of the Server

Create a file called **app.py** in your project root. This is the main server file. Let's build it piece by piece:

Part A: Imports and Setup

```python
import os, json, uuid
from flask import Flask, request, jsonify, send_from_directory
from flask_socketio import SocketIO, emit, join_room
from pymongo import MongoClient
from pymongo.server_api import ServerApi

app = Flask(__name__, static_folder="public", static_url_path="")
app.config["SECRET_KEY"] = "ai-catalyst-secret"
socketio = SocketIO(app, cors_allowed_origins="*")

DB_PATH = os.path.join(os.path.dirname(__file__), "data", "db.json")
MONGODB_URI = os.environ.get("MONGODB_URI")  # Set in Render dashboard
```

This sets up Flask to serve your website files from the **public/** folder and enables WebSocket support for real-time features. The **MONGODB_URI** environment variable connects to MongoDB Atlas in production.

Part B: Database Helpers (with MongoDB)

PYTHON

```python
# Lazy MongoDB connection (see Chapter 3)
_mongo_db = None

def _get_mongo():
    global _mongo_db
    if _mongo_db is None and MONGODB_URI:
        client = MongoClient(MONGODB_URI,
                             server_api=ServerApi("1"))
        _mongo_db = client["aicatalyst"]
    return _mongo_db

def read_db():
    mongo = _get_mongo()
    if mongo:  # Use MongoDB in production
        db = {}
        for col in ["config","members","events",
                    "points","quizzes","surveys","teams"]:
            docs = list(mongo[col].find({}, {"_id": 0}))
            db[col] = docs[0] if col == "config" else docs
        return db
    with open(DB_PATH) as f:  # Fallback to local file
        return json.load(f)

def write_db(data):
    mongo = _get_mongo()
    if mongo:
        for col, val in data.items():
            mongo[col].delete_many({})
            items = [val] if col == "config" else val
            if items:
                mongo[col].insert_many(items)
    with open(DB_PATH, "w") as f:
        json.dump(data, f, indent=2)
```

■ Why Two Functions? (Now with MongoDB!)

read_db() first checks if MongoDB Atlas is available. If yes, it reads from the cloud database. If not (e.g., running locally without MONGODB_URI), it falls back to the db.json file. write_db() saves to BOTH MongoDB and db.json — so your local file stays in sync. This dual-write approach means you always have a backup!

Part C: API Routes (CRUD)

API routes are URLs that the frontend calls to get or change data. We follow a pattern called **CRUD** — the four basic operations on any data:

Operation	HTTP Method	URL Pattern	What It Does
Create	POST	/api/members	Add a new member

Read	GET	/api/members	List all members
Update	PUT	/api/members/:id	Change a member's info
Delete	DELETE	/api/members/:id	Remove a member

This same CRUD pattern repeats for **every resource**: events, points, quizzes, surveys, and teams. Once you understand it for members, you understand it for all!

```python
@app.route("/api/members", methods=["GET"])
def get_members():
    db = read_db()
    return jsonify(db["members"])

@app.route("/api/members", methods=["POST"])
def create_member():
    db = read_db()
    data = request.json
    name = data.get("name", "")
    member_id = name.lower().replace(" ", "-")
    new_member = {"id": member_id, **data}
    db["members"].append(new_member)
    write_db(db)
    return jsonify(new_member), 201
```

How a Single Click Travels Through the System

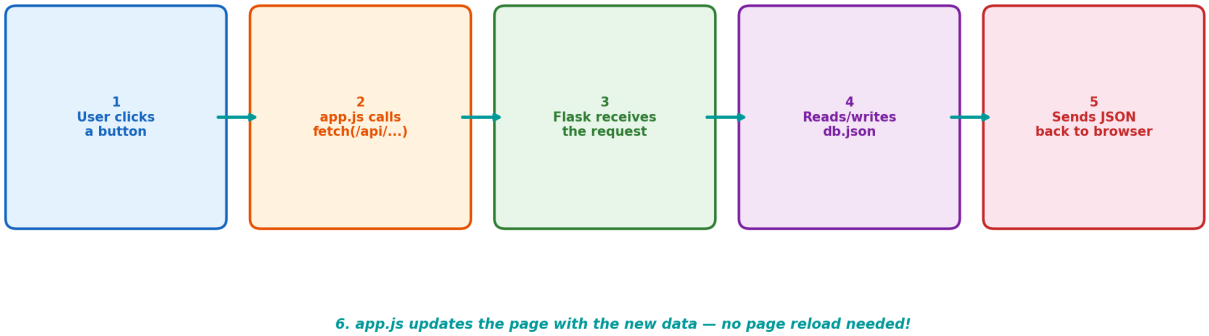

Figure 4.1 — The journey of a single click through the system.

Part D: Starting the Server

```python
if __name__ == "__main__":
    port = int(os.environ.get("PORT", 3000))
    socketio.run(app, host="0.0.0.0", port=port, debug=True)
```

When you run **python app.py**, this starts the server on port 3000. Open your browser and go to **http://localhost:3000** to see your site!

5 Building the Frontend Interface

■ The Building Analogy

Think of the frontend like building a house: • HTML = The structure (walls, rooms, doors) — defines WHAT is on the page • CSS = The paint and decoration — defines how it LOOKS • JavaScript = The electricity and plumbing — defines how it WORKS

Step 1: Create the folder structure

TERMINAL

```
mkdir public
mkdir public/js
mkdir public/css
mkdir uploads
mkdir uploads/avatars
```

Project Folder Structure

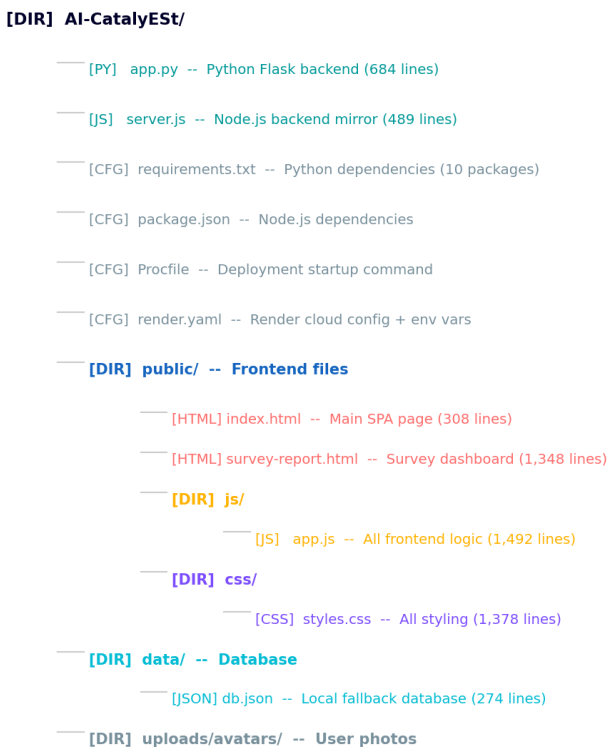

Figure 5.1 — Complete project folder structure with file purposes and line counts.

Step 2: Create index.html — The One and Only Page

Our entire dashboard uses a technique called **Single-Page Application (SPA)**. Instead of having separate pages for dashboard, leaderboard, calendar, etc., we have ONE HTML file with ALL sections. JavaScript shows/hides them like tabs.

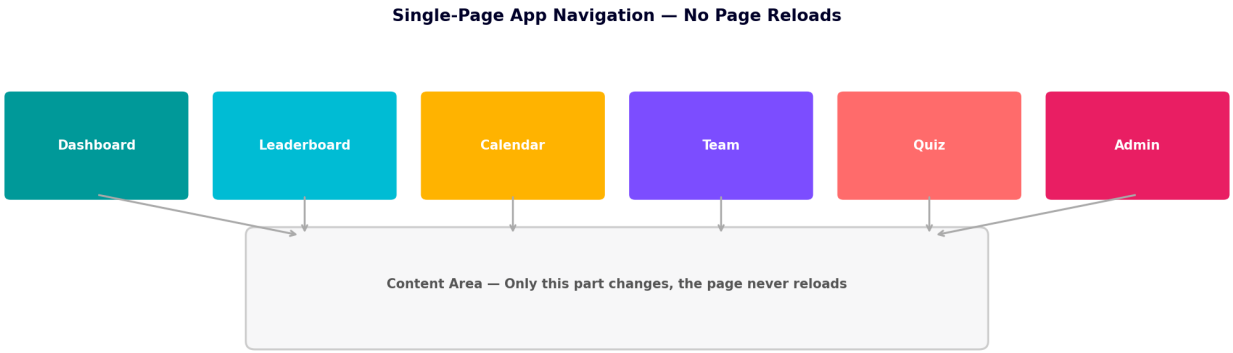

Figure 5.2 — SPA navigation: clicking tabs shows different content without reloading the page.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
  <title>AI CatalyESt Dashboard</title>
  <link rel="stylesheet" href="/css/styles.css">
</head>
<body>
  <!-- Navigation Bar -->
  <nav id="main-nav">...</nav>

  <!-- Each tab is a <section> -->
  <section id="tab-dashboard">...</section>
  <section id="tab-leaderboard">...</section>
  <section id="tab-calendar">...</section>
  ...

  <!-- Scripts loaded LAST (after page renders) -->
  <script src="/js/app.js"></script>
</body>
</html>
```

■ Why Load Scripts at the Bottom?

When the browser reads your HTML from top to bottom, if it hits a <script> tag, it pauses to download and run that script. By putting scripts at the BOTTOM, the user sees the page structure immediately while scripts load in the background. This prevents the "blank screen" problem.

Step 3: Create app.js — The Brain of the Frontend

This is the largest file in the project (1,234 lines). It controls everything the user interacts with. Here's the key pattern:

JAVASCRIPT

```
// 1. GLOBAL STATE — stores all data in memory
const state = {
  members: [],
  events: [],
  points: [],
  leaderboard: [],
  currentTab: "dashboard",
};

// 2. API HELPER — talks to the server
async function api(url, options = {}) {
  const res = await fetch(url, { ...options });
  return await res.json();
}

// 3. LOAD DATA — fetches everything when page opens
async function loadAllData() {
  state.members = await api("/api/members");
  state.events = await api("/api/events");
  // ... etc
}

// 4. RENDER FUNCTIONS — build HTML for each tab
function renderDashboard() {
  document.getElementById("tab-dashboard").innerHTML = `
    <h1>Welcome to AI CatalyESt</h1>
    <p>${state.members.length} members</p>
  `;
}
```

Every tab has its own **render function**: renderDashboard(), renderLeaderboard(), renderCalendar(), renderTeam(), etc. When you click a tab, JavaScript calls the corresponding render function, which generates HTML from the data in **state**.

6 Adding Real-Time Features

■ Phone Call vs. Letters

Normal web requests (HTTP) are like sending letters — you send one, wait for a reply, done. WebSockets are like a phone call — once connected, both sides can talk freely in real-time. This is how our quiz system works: the server can PUSH questions to all players simultaneously.

Socket.IO — WebSockets Made Easy

We use a library called **Socket.IO** that handles WebSocket connections for us. It works on both the server (Python) and client (JavaScript) sides.

Server Side (Python):

PYTHON

```python
@socketio.on("quiz:join")
def handle_join(data):
    """When a player joins a quiz room"""
    room = data["quizId"]
    join_room(room)
    emit("quiz:playerJoined",
         {"player": data["name"]},
         room=room)
```

Client Side (JavaScript):

JAVASCRIPT

```javascript
// Connect to the server
const socket = io();

// Listen for questions
socket.on("quiz:question", (data) => {
  showQuestion(data.question);
});

// Send an answer
socket.emit("quiz:answer", {
  quizId: "quiz-abc123",
  answer: 2  // option index
});
```

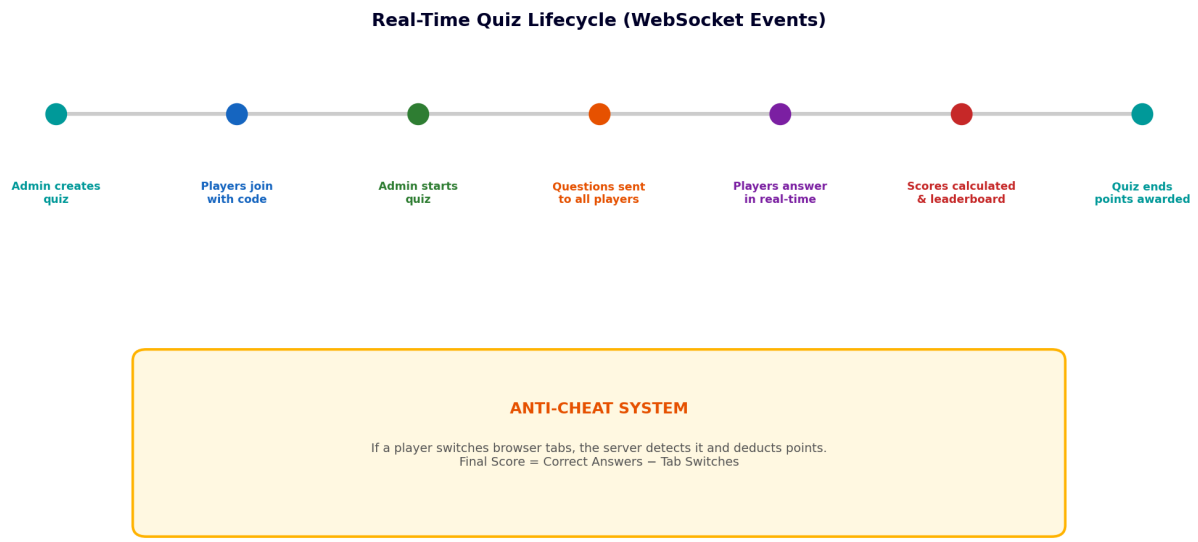

Figure 6.1 — Complete quiz lifecycle from creation to scoring, including anti-cheat detection.

■■ Anti-Cheat System

The quiz has a clever anti-cheat mechanism. If a player switches to another browser tab (presumably to Google the answer), the browser fires a "visibilitychange" event. Our JavaScript catches this and tells the server. The server tracks tab switches per player and deducts points: Final Score = Correct Answers – Tab Switches.

7 Styling & Visual Design

The styling file (styles.css, 1,207 lines) gives the dashboard its Siemens-branded, Apple-like minimalist look. Let's understand the key design decisions.

CSS Custom Properties (Variables)

Instead of typing the same color code everywhere, we define variables once and use them throughout. If we want to change the brand color, we change it in ONE place.

```css
:root {
  --primary:    #009999;   /* Siemens Petrol */
  --dark:       #000028;   /* Deep Navy */
  --green:      #00FFB9;   /* Bright Accent */
  --bg:         #F7F7F8;   /* Light Background */
  --font:       "Inter", sans-serif;
  --radius:     16px;      /* Rounded corners */
  --shadow:     0 2px 20px rgba(0,0,0,0.06);
}
```

#009999	#000028	#00FFB9	#00BEDC	#F7F7F8
Petrol	Dark Navy	Green	Light Blue	Background
Primary	Text	Accent	Secondary	Surface

Figure 7.1 — The Siemens-inspired color palette used throughout the dashboard.

Design Principles Applied

- **Generous Whitespace** — Elements have ample padding (20-32px) and margins so nothing feels cramped.
- **Rounded Corners** — All cards use 16px border-radius for a soft, modern feel.
- **Subtle Shadows** — Light box-shadows give depth without heaviness.
- **Consistent Typography** — Inter font with a clear hierarchy: 28px for titles, 13px for body text.
- **Responsive Design** — CSS media queries adjust layout for phones, tablets, and desktops.
- **Smooth Animations** — Hover effects, tab transitions, and card entries all use CSS transitions.

8 Testing Everything Locally

Running the Server

TERMINAL

```
# Make sure your virtual environment is active!
# You should see (.venv) in your prompt.

python app.py

# You should see:
# * Running on http://0.0.0.0:3000
# * Debugger is active!
```

1 Open your browser

Navigate to http://localhost:3000 — you should see the dashboard with the AI CatalyESt header.

2 Test the Dashboard tab

The countdown timer should show time remaining until the next event. The activity cards should display.

3 Test Admin Login

Click the login icon in the navigation bar. Enter username: admin, password: admin. After login, you should see additional admin buttons appear.

4 Test Adding a Member

Go to Admin > Members. Click "Add Member". Fill in the form and submit. Check the Team tab to verify the new member appears.

5 Test a Live Quiz

Go to Admin > Quizzes. Create a quiz with 2-3 questions. Start it. Open a second browser window to join as a player.

■ Checkpoint ✓

If all 5 tests above work correctly, your application is ready for deployment! If something doesn't work, check the terminal for error messages — Python will tell you exactly what line caused the problem.

Common Issues During Testing

Issue	Cause	Fix
Page is blank	Server not running	Run python app.py in terminal
Port already in use	Another app on port 3000	Kill it or change PORT=3001
"Module not found"	Missing dependency	Run pip install -r requirements.txt
Data not loading	db.json has invalid JSON	Validate at jsonlint.com
Admin buttons missing	Not logged in	Login with admin/admin

9 Version Control with GitHub

■ The Video Game Analogy

Git is like a save system in a video game. Every time you "commit", you create a save point. If you mess something up, you can go back to any previous save. GitHub is like cloud storage for your saves — it keeps them safe online and lets others access them.

Step 1: Create a GitHub Account

Go to **github.com** and sign up for a free account. Choose a username you'll remember — it becomes part of your repository URL.

Step 2: Install GitHub CLI

TERMINAL

```
# Windows (using winget):
winget install GitHub.cli

# Then login:
gh auth login
# Follow the prompts — choose "GitHub.com" and "HTTPS"
```

Step 3: Initialize Git in Your Project

TERMINAL

```
# Inside your project folder:
git init

# Create a .gitignore file to exclude unnecessary files:
# (files like .venv/, node_modules/, __pycache__/)
```

The **.gitignore** file tells Git which files NOT to track. You don't want to upload your virtual environment (it's huge) or temporary files.

TEXT

```
# .gitignore contents:
.venv/
node_modules/
__pycache__/
*.pyc
uploads/avatars/*
!uploads/avatars/.gitkeep
```

Step 4: Create Repository and Push

TERMINAL

```
# Create a new repository on GitHub:
gh repo create ai-catalyst --public --source=.

# Add all files, create a save point, and upload:
git add -A
git commit -m "Initial commit: AI CatalyESt dashboard"
git push origin master
```

From Your Computer to the Internet — Deployment Pipeline

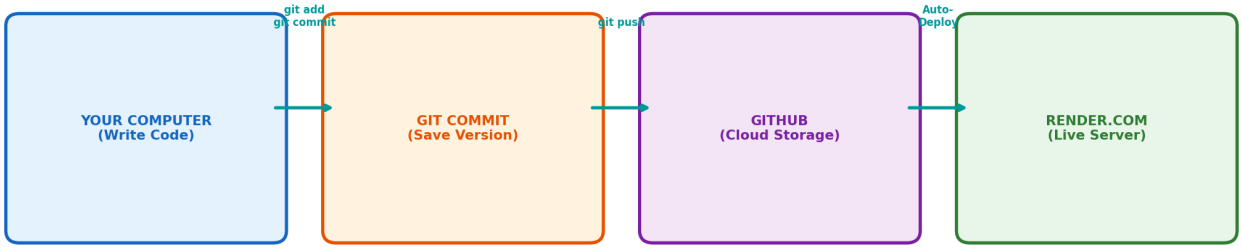

Figure 9.1 — The complete deployment pipeline from your computer to the live internet.

■ Golden Rule of Git

Commit early, commit often! Every time you complete a small piece of work (add a feature, fix a bug), create a commit. Your commit message should describe WHAT changed, like: "Add leaderboard sorting" or "Fix quiz timer countdown".

10 Deploying to the Cloud

Right now your dashboard only works on your computer (localhost). To make it accessible to anyone on the internet, we need to put it on a **cloud server**. We'll use **Render.com** — a free hosting platform.

> ■ **What is "Deploying"?**
> Deploying means copying your code to a computer on the internet (a server) that runs 24/7, so anyone with the URL can access your website. It's like moving your restaurant from your kitchen to a real storefront that customers can visit anytime.

Step 1: Create Configuration Files

Render needs to know how to start your server. Create these files:

Procfile — tells Render the startup command:

```
web: gunicorn -k geventwebsocket.gunicorn.workers.GeventWebSocketWorker -w 1 --bind 0.0.0...
```
TEXT

runtime.txt — tells Render which Python version:

```
python-3.12.3
```
TEXT

render.yaml — full service configuration:

```yaml
services:
  - type: web
    name: ai-catalyest
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn -k geventwebsocket...
    envVars:
      - key: PYTHON_VERSION
        value: 3.12.3
      - key: RENDER
        value: true
```
YAML

Step 2: Set Environment Variables on Render

Environment variables are secrets and settings that your code reads at runtime. They keep sensitive data (like database passwords) OUT of your code. Set these in the Render dashboard under your

service's "Environment" tab:

Variable	Value	Purpose
MONGODB_URI	mongodb+srv://user:pass@cluster...	Connects to MongoDB Atlas cloud database
PYTHON_VERSION	3.12.3	Tells Render which Python to use
RENDER	true	Lets the app detect it is running on Render

■ Never Commit Secrets!

The MONGODB_URI contains your database password. NEVER put it in your code or push it to GitHub. Always use environment variables (set in the Render dashboard) to keep secrets safe. Our code reads it with: os.environ.get("MONGODB_URI")

Step 3: Sign Up on Render.com

1. **Go to render.com**
   Click "Get Started for Free" and sign up with your GitHub account.

2. **Create New Web Service**
   Click "New +" → "Web Service". Connect your GitHub repository.

3. **Configure Settings**
   Name: ai-catalyest. Runtime: Python 3. Build: pip install -r requirements.txt.

4. **Deploy!**
   Click "Create Web Service". Render will build and deploy your app in about 3 minutes.

Step 4: Auto-Deploy from GitHub

The best part: Render watches your GitHub repository. Every time you push code to GitHub, Render automatically deploys the update. Just push and wait 2-3 minutes!

TERMINAL

```
# Make a change, then:
git add -A
git commit -m "Update dashboard styling"
git push origin master

# Render auto-deploys in ~2-3 minutes!
```

■■ Free Tier Limitation

Render's free plan puts your server to "sleep" after 15 minutes of no visitors. The first visit after sleep takes ~30 seconds to "wake up". This is normal! Our code handles this with a loading overlay and auto-retry logic.

11 Project Analytics & Insights

Let's step back and look at the complete project from a data perspective. These charts reveal the scope, complexity, and composition of what we built.

Total Lines of Code	Project Files	API Endpoints	Team Members
9,700+	**18+**	**20+**	**16**
Across 5 languages	Python, JS, HTML, CSS, JSON	REST + WebSocket events	In 5 domains

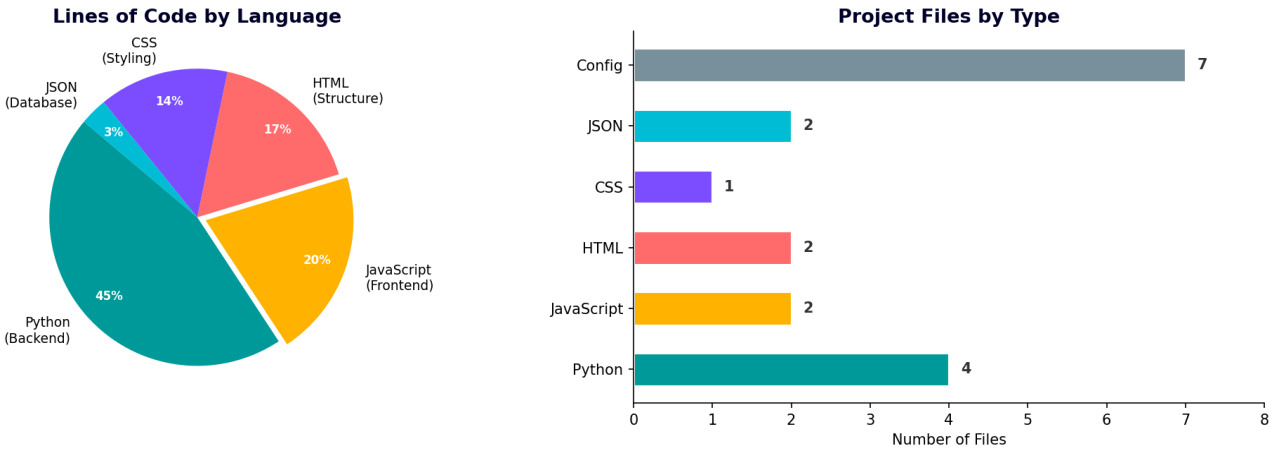

Figure 11.1 — Code distribution by language and file count breakdown.

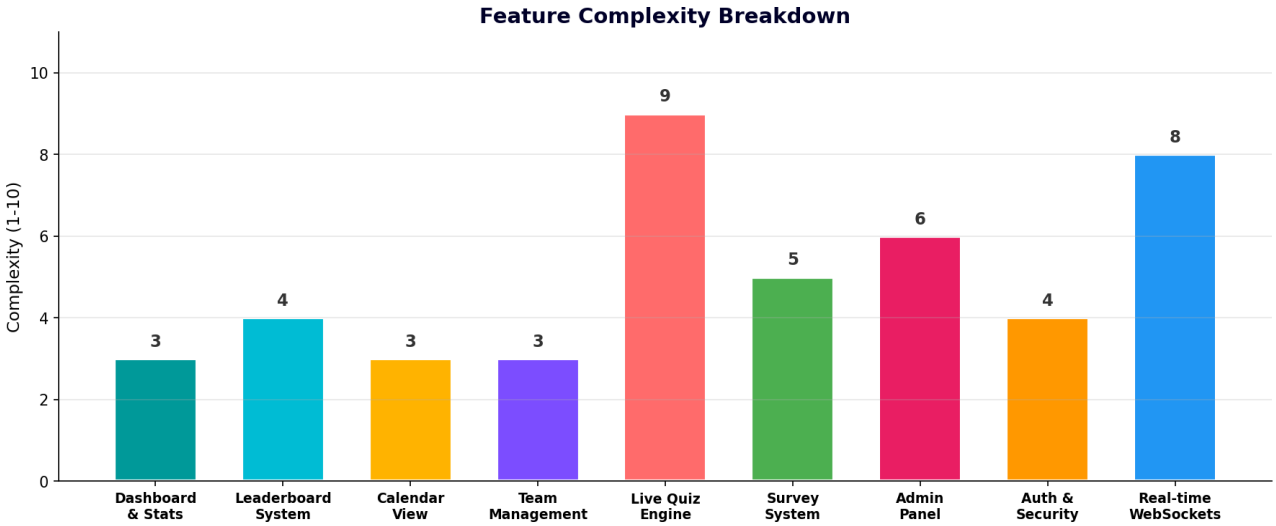

Figure 11.2 — Relative complexity of each feature. The live quiz engine and WebSocket system are the most complex.

Technology Choices & Why

Technology	Role	Why We Chose It
Python + Flask	Backend server	Easy to learn, minimal boilerplate, great for APIs
Socket.IO	Real-time comms	Handles WebSockets with automatic reconnection
Vanilla JS	Frontend logic	No framework complexity — simple and fast
MongoDB Atlas	Cloud database	Free M0 tier, JSON-like documents, auto-scales
JSON file	Local fallback DB	No setup needed — fallback if MongoDB is unavailable
CSS Variables	Theming	One-place brand color changes, maintainable design
Render.com	Hosting	Free tier, auto-deploy from GitHub, supports Python
GitHub	Version control	Free, industry standard, enables auto-deploy

? Troubleshooting Guide

Problem: The page shows "Not Found" on a white screen

■ Cause

The server is not running, or it's a cold start on Render's free tier.

■ Solution

Wait 30 seconds and refresh. The loading overlay should appear and auto-retry. If running locally, make sure python app.py is running in your terminal.

Problem: The page loads with huge/disproportionate text

■ Cause

External fonts (Google Fonts) or CSS haven't loaded yet.

■ Solution

This is handled by the loading overlay we added. It shows a spinner until everything loads. The inline critical CSS ensures the overlay itself always renders correctly.

Problem: Data is not loading (empty dashboard)

■ Cause

API calls are failing — either the server isn't ready or db.json is corrupted.

■ Solution

Check the browser console (F12 → Console tab) for error messages. Validate your db.json at jsonlint.com. Our api() function retries 3 times automatically.

Problem: "ServerSelectionTimeoutError" or MongoDB connection fails

■ Cause

The MONGODB_URI environment variable is missing or incorrect, or your IP is not whitelisted.

■ Solution

Check MONGODB_URI in Render's Environment tab. In MongoDB Atlas, go to Network Access and make sure 0.0.0.0/0 is allowed. The app will fall back to db.json if MongoDB fails.

Problem: "ModuleNotFoundError: No module named flask"

■ Cause

Python packages are not installed or virtual environment is not active.

■ Solution

Run: pip install -r requirements.txt. Make sure (.venv) appears in your terminal prompt.

Problem: Changes not appearing on the live site

■ Cause

You forgot to push to GitHub, or Render hasn't finished deploying.

■ Solution

Run: git add -A && git commit -m "update" && git push. Then wait 2-3 minutes for Render.

Problem: Quiz participants can't join

■ Cause

WebSocket connection failed — possibly a firewall or CORS issue.

■ Solution

Make sure you're using the correct URL. On Render, use https:// not http://.

Problem: Admin buttons are not visible

■ Cause

You're not logged in as admin.

■ Solution

Click the lock icon → enter admin/admin → admin buttons will appear in the nav bar.

G Glossary of Terms

These are the technical terms used in this manual, explained in plain English.

API (Application Programming Interface)

A set of rules that lets two programs talk to each other. Our frontend uses APIs to ask the backend for data, like asking a librarian for a specific book.

Backend

The "behind-the-scenes" part of a website — the server and database that users never see directly. Like the kitchen in a restaurant: essential but hidden.

Browser

The application you use to visit websites — Chrome, Firefox, Edge, Safari. It reads HTML/CSS/JS and turns them into the visual page you see.

Commit

A "save point" in Git. Each commit captures the state of all your files at that moment, with a message describing what changed.

CRUD

Create, Read, Update, Delete — the four basic operations you can do on any data. Every resource in our API supports these four operations.

CSS (Cascading Style Sheets)

The language that controls how a webpage LOOKS — colors, fonts, spacing, animations. It's the paint and wallpaper of your house.

Database

Where data is stored permanently. We use MongoDB Atlas (cloud) as the primary database and db.json (a local JSON file) as a fallback.

Deployment

The process of putting your website on a server so anyone on the internet can access it.

Flask

A Python framework (pre-built code library) that makes it easy to create web servers. It handles HTTP requests, URL routing, and more.

Frontend

The part of a website that users see and interact with — the HTML page, styles, and JavaScript. Like a restaurant's dining area with menus and tables.

Git

A version control system that tracks changes to your files over time. Think of it as an "unlimited undo" button for your entire project.

GitHub

A website that hosts Git repositories online. Like Google Drive but specifically for code. Free to use, and millions of developers use it.

HTML (HyperText Markup Language)

The language that defines the STRUCTURE of a webpage — headings, paragraphs, buttons, images. It's the skeleton/frame of your house.

HTTP (HyperText Transfer Protocol)

The "language" browsers and servers use to communicate. When you type a URL, your browser sends an HTTP request. The server sends back an HTTP response.

JavaScript

The programming language of the web. It makes pages interactive — handles clicks, animates elements, and communicates with servers. The electricity of your house.

JSON (JavaScript Object Notation)

A text format for storing data as labeled pairs: {"name": "Vigneshvar", "age": 22}. Easy for both humans and computers to read.

Render.com

A cloud platform that hosts websites. It connects to your GitHub repository and automatically deploys your code to the internet.

REST (Representational State Transfer)

A design pattern for APIs where each URL represents a resource and HTTP methods (GET, POST, PUT, DELETE) represent actions on it.

MongoDB Atlas

A free cloud-hosted NoSQL database service. Stores data as JSON-like "documents" instead of rows and columns. Our production database runs on MongoDB Atlas M0 (free tier).

Environment Variable

A setting stored outside your code (on the server), not in files. Used for secrets like database passwords. Read in Python with os.environ.get("VARIABLE_NAME").

SPA (Single-Page Application)

A web app that loads ONE HTML page and dynamically updates content using JavaScript, instead of loading new pages. Feel faster and smoother.

Socket.IO / WebSocket

Technology for real-time, two-way communication between browser and server. Like a phone call instead of exchanging letters.

Terminal / Command Line

A text-based interface where you type commands to control your computer. Every developer uses it to run programs, install packages, and manage files.

Virtual Environment

An isolated Python workspace. Like a separate toolbox for each project so their dependencies don't interfere with each other. Created with python -m venv .venv.

You've Reached the End!

Congratulations on reading through the entire manual. You now have the knowledge to build and deploy a full-stack web dashboard from scratch — no AI assistance needed.

Live Dashboard: https://ai-catalyest.onrender.com

Source Code: github.com/vigneshvar11/ai-catalyst

Built with ❤■ by Vigneshvar SA | Siemens Technology India | Engineering Systems

Manual generated on May 10, 2026